Self-Driving-Car-Simulation

1. The Problem we are solving:

Modern autonomous vehicles (AVs) rely almost exclusively on deep learning and neural networks to navigate the world. While these "black-box" models are excellent at pattern recognition (like identifying a stop sign), they struggle with logical reasoning and explainability. When a neural network makes a catastrophic error, it is incredibly difficult for engineers to determine exactly *why* the decision was made. Furthermore, these systems often fail in rare "edge cases" that they haven't explicitly been trained on, lacking the common-sense, heuristic reasoning that human drivers use.

2. Why It Matters:

The lack of transparency in current AV technology is a major hurdle for public trust, legal accountability, and ultimate safety. If a self-driving car is involved in an accident, investigators, insurance companies, and the public need to know the exact decision-making process that led to the event. Creating an **Explainable AI (XAI)** system where every action can be traced back to a specific, understandable rule is critical for the safe integration of autonomous vehicles into society.

3. Approach to Solving It:

To address the "black-box" problem, this project shifts away from machine learning and utilizes **Symbolic AI** built in Prolog. The approach involved building a rule-based expert system that mimics human deductive reasoning.The environment was abstracted into a series of **dynamic facts** (e.g., current speed, distance to obstacles, traffic light status, and weather conditions). The car's "brain" was programmed using **declarative logic rules**—a strict hierarchy of "If-Then" conditions representing traffic laws and safety heuristics. Instead of guessing based on probability, the car queries its knowledge base and uses logical deduction to guarantee a safe maneuver based on the current state of its world.

4. Key Decisions Made:

- **Choosing Prolog:** I opted for Prolog (a declarative logic programming language) rather than a traditional imperative language like Python or C++. This allowed me to define *what* the rules of the road are, rather than writing complex, nested if/else loops dictating *how* the car should evaluate them.
- **Priority-Based Rule Architecture:** I structured the decision-making engine decide_action/1 strictly by priority. Life-threatening situations (Priority 1: Emergency Braking) are evaluated before traffic laws (Priority 2: Red Lights), which are evaluated before efficiency rules.

- **Implementing the Prolog "Cut" (!):** I utilized the cut operator to optimize the engine and prevent backtracking once a critical safety decision was reached. This ensures the car commits to an emergency brake and doesn't confusingly try to evaluate standard driving rules simultaneously.
- **State Management:** I decided to simulate the passage of time and changing environments by using Prolog's dynamic memory allocation (assertz and retractall) to continuously wipe and rewrite the sensor data for every simulation step.
-

5. Challenges Faced:

- **Rule Conflicts:** Early in development, the car would encounter scenarios where multiple rules were technically true (e.g., the light is green, but there is a pedestrian in the road). Resolving this required careful restructuring of the knowledge base to ensure high-priority safety constraints naturally overrode standard driving permissions.
- **Managing Dynamic State:** Prolog is inherently designed for static knowledge querying, not real-time state changes. Figuring out how to safely update the car's "memory" of its speed and surroundings without breaking the logic engine required learning how to properly handle dynamic predicates.
- **Simulating Weather Impacts:** Introducing variables like rain required expanding the logic beyond simple distance checks. I had to build conditional logic that dynamically altered the acceptable speed limits based on a secondary environmental factor.

6. What I Learned:

Through this project, I gained a deep appreciation for the **Declarative Programming Paradigm**. I learned how to translate messy, real-world human heuristics (like deciding when to yield at a flashing light) into strict, formal mathematical logic.

More importantly, this project highlighted the immense value of **Explainable AI**. By building an autonomous system where every output is accompanied by a transparent trail of logical deductions, I saw firsthand how Symbolic AI can solve the trust and accountability issues currently plaguing modern machine learning approaches.